Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 160 (2019) 778–784

Procedia Computer Science

www.elsevier.com/locate/procedia

The International Workshop on Emerging Networks and Communications
(IWENC 2019)
November 4-7, 2019, Coimbra, Portugal

Detection of Timestamps Tampering in NTFS using Machine Learning

ALJI Mohamed[a], CHOUGDALI Khalid[a,*]

[a] *Electronics and Telecommunication Systems Research Group*
National School for Applied Sciences, Ibn Tofail University
Kenitra, Morocco

Abstract

During a digital investigation, the recorded time of activity on the system is crucial for solving the case. But file times may be subject to user manipulation for deceptive reasons. Detecting such timestamps change, in a none-automatic way, will come to finding a needle in a haystack. Many ways can lead to timestamps manipulation: the presence of anti-forensics tools, unusual timestamp differences in the volume shadow copies, the system restore points and the filesystem metadata, inconsistencies in the filesystem timestamps or with the established rules of normal time behavior, timeline analysis, etc. However, while reviewing the literature, we found little use of the capabilities of machine learning algorithms in such detection. In this paper, a machine learning approach for the automatic detection of timestamps tampering is proposed to reduce the required manual search for such manipulation. Put differently, the approach predicts a classification of input files in whether they have been timestamp tampered or not. Furthermore, the process of a synthetic dataset collection, features engineering and extraction, dataset manipulation, training, and model evaluation is presented. To recapitulate, the held experiment generates the synthetic dataset from a virtual controlled environment, apply a machine learning algorithm on a subset of the dataset, predict on the other subset of the dataset and present the results using confusion matrix, receiver operating characteristic curves, precision-recall curves, accuracy, and log_loss.

© 2019 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the Conference Program Chairs.

Keywords:
Machine learning; Logistic Regression; Detection; Timestamps manipulation; NTFS; Digital investigation.

1. Introduction

In today's world, we rely heavily on the information stored on computers. The question can we trust the information and the details or the metadata that comes with it. For instance, a none-well-intentioned computer user may seek to

* Corresponding author. Tel.: +212.6.58.69.44.58.
E-mail addresses: mohamed.alji@uit.ac.ma (ALJI Mohamed)., chougdali@gmail.com (CHOUGDALI Khalid).

1877-0509 © 2019 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the Conference Program Chairs.
10.1016/j.procs.2019.11.011

modify or edit time information stored on the metadata of a file such as the date and the time of a photo taking or a stored file on the drive. A cyber-criminal may seek to hide its digital footprints to escape justice, and in a digital investigation, it is up to the forensic experts to ensure the validity, to a certain degree, of the information presented to the court of law. We conclude that a computer user may at any time engage an anti-forensic tool such as timestamps manipulator. Such tools use a technique that consists of tampering or changing the timestamps in files metadata for a deceptive reason. To show the importance of file timestamps, let consider the following case:

A computer user creates and edit a text file and then changes its timestamps to a future day X using a time stomper tool. The purpose of changing file times is to fabricate an alibi for that specific day. Then, he commits some illegal activity during the X day. Summoned as a suspect in front of the court of law, he insisted on having an alibi at the period time of the day X. Indeed, the text file times show that he was working on that text file. It is up to the digital forensic expert to confirm or refute the validity of the alibi [1].

In this study, we held an experiment where we collected and prepared a time information dataset from a disk image formatted using NTFS filesystem. We managed to select a machine learning algorithm that automates the detection of the tampered timestamps in a controlled environment. Finally, we evaluate the results for validation purposes. In other words, we are approaching the problem of automatically detecting the timestamps manipulation in NTFS filesystem from the data-driven perspective rather than from the rule-based perspective.

1.1. Related work

As stated before, we have found little use of the machine learning algorithms in the detection of timestamps tampering in NTFS filesystem. So, we mainly focused on reviewing the literature for the detection of timestamps manipulation in NTFS filesystem not necessary using a machine learning algorithm. The results are as follows:

The study [2, 3] has verified in different Windows OS versions that the changes in the time information stored in the attributes `$Standard_Information` (`$SI`) and `$Filename` (`$FN`) of the filesystem are the results of different user behaviors such as Copy, Move, and Deletion of files or folders. It was found to be possible to detect many kinds of user manipulations by comparing the time information in these attributes. Another study [1] has utilized the special NTFS journaling file `$LogFile` to detect timestamp forgeries. The Redo/Undo attribute of Log records may contain past-and-present valuable forensic time information. The research paper [4] has presented a triage methodology for automating the categorization of digital media using machine learning. The methodology was applied in two use cases: copyright infringement and child pornography exchange to prove its viability. Other related work [5, 6] have listed more normal user patterns that change the stored time information, such as copy, move, create, change properties, internet download, program install, etc. Following the selected patterns, the latter study deduced that the timestamps changes in case of a normal user behavior follow 7 rules. Based on those rules, the detection of anomalies (timestamps tampering) is possible. The study [7] has suggested that the redundant information on filesystem can be used for information hiding and had proposed a countermeasure for detecting the hidden information. Other study [8] has observed that file metadata and timestamps across different cloud (Ubuntu ext4 and Windows NTFS) and has compared the access behavioral patterns from the time information perspective. The research paper [9] has proposed a methodology for the automatic classification of suspicious file artifacts using supervised machine learning to solve the challenge of detecting quickly pertinent file artifacts to a digital investigation.

After a general overview in the first section (1), section 2 describes the proposed machine learning approach. We outline briefly each step of the designed experiment from the environment set-up, and dataset preparation, model building, to the model evaluation. For each selected evaluation metric, we present and discuss in section 3 the experimental results. Finally, we conclude the article by showing the limitations of our approach and future research work.

2. Methodology

Traditionally, the presence of an anti-forensics tool such as `Timestomp`, or incoherences in timeline analysis may suggest the manipulation of timestamps, and by so, inciting for further investigation. This research aims at reducing the manual effort of looking for a possible timestamps manipulation. The manual effort could be searching for the most common time stompers that may have been used by the suspect or digging deeper into the huge amount of data that

may be generated by the timeline analysis process. This paper presents an automatic and a different way of detecting timestamps tampering in NTFS filesystem using supervised machine learning. Figure 1 illustrates the flow-chart of the designed approach from the input disk image, through generation and manipulation of the dataset, to the learning, prediction and the evaluation of the predictive capabilities of the built model. The process is broken down into six steps and each of the following subsection corresponds to a step of the process.

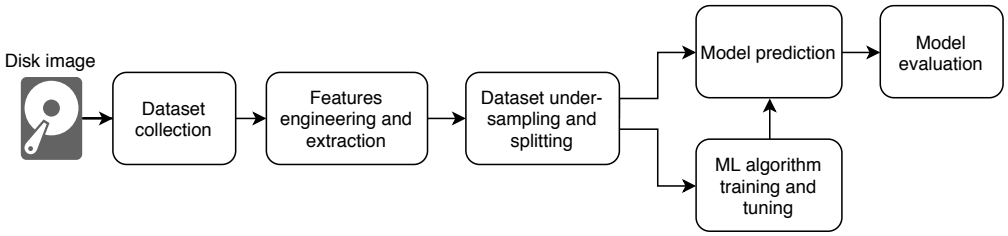


Fig. 1. Flow chart of the proposed approach.

2.1. Dataset collection

Dataset collection consists of the generation of the needed data for the experiment. The data here are a group of files with their timestamps tampered using random time values. In order to create those files, we set-up a Windows 10 Pro 'v1809Oct' virtual machine on our working computer that is running `Debian GNU/Linux 9` as a host OS with the following capacity: 8 GiB (RAM), Intel Core i3-3110M CPU 2.40GHz x 4 (Processor), SSD 84.4 GiB (dedicated disk partition). Virtualization is done using Oracle VirtualBox Manager and provides the capability of rolling back the virtual machine to an old state. In the virtual machine, we dedicated a virtual disk partition for the guest OS and another virtual disk partition for the manipulated timestamps files.

In order to generate the data efficiently, we used PowerShell scripting. The written script loops while creating an important number of text files, adding some content to them, and then applying a timestamps manipulator tool like those available online: `Timestomp`, `SetMace` or `Change Timestamp` on those text files. The results are the files with some of their timestamps tampered using random time values. Since the data generated at this step is meant to be training and testing sets for the machine learning model, we made sure each use of the timestamps manipulator represents a possible use in a real case. In another way, if ever the tool can make 5 possible changes (Modified, Accessed, Created and MFT Entry modified and all at once), we have to take in consideration all those 5 cases.

2.2. Feature vectors engineering and extraction

NTFS is a filesystem that organizes the files in the hard disk using some special files, such as the Master File Table `$MFT` [10]. The NTFS `$MFT` file stores information about each file on the system in a similar way as a table structure where columns are attributes of files and rows are pointers to files (for simplification). Among the NTFS `$MFT` attributes, there are `$SI` and `$FN` of interest that contain useful meta-information about the files such as the filename, the extension, the timestamps, etc.

We developed a script that uses `VBoxManage` to convert the Virtual Disk Image (VDI) of the virtual machine into raw disk image. The script uses `pytsk` [1] (a Python bindings for the SleuthKit). It extracts the NTFS `$MFT` file of each NTFS partition in the provided `raw` disk image. Using `dfir_ntfs`[2] module, we convert the NTFS `$MFT` files into a comma-separated format (`csv`). Figure 2 illustrates the work-flow of the process of data extraction from the virtual machine to the file in format `csv`.

The program then uses the `Pandas`[3] library to load the content of the `csv` files into memory. While loading, the program parses the date and time values into its corresponding type within the limitations of the Pandas timestamp

[1] https://github.com/py4n6/pytsk
[2] https://github.com/msuhanov/dfir_ntfs
[3] https://pandas.pydata.org

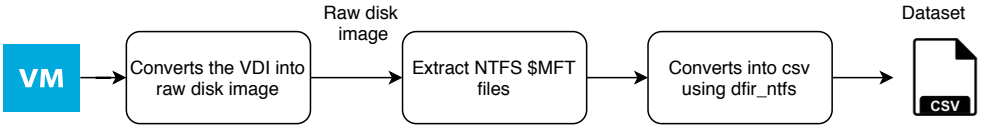


Fig. 2. Work flow of the data extraction and processing

implementation as provided on their official documentation. The program handles the missing time data or not-a-number cases by setting an edge value to them.

The features of interest are the timestamps extracted from the NTFS `$MFT` files. The timestamps are transformed into numerical features. Timestamp, in their current format, are not suitable for use in machine learning model, so we extracted from each timestamp the following time information: Years, Months, Days number, Hours, Minutes, Seconds, Microseconds. Each feature has been manipulated to reside within the numeric scale of [0, 1].

2.3. Dataset under-sampling and splitting

The visualization of the dataset records grouped by class shows that it is highly imbalanced (Figure 3). Indeed, the timestomped text files are few against the none-tampered rest of the system files. So, we adopted a down-sampling strategy to rebalance the dataset.

Since we are limited in term of the synthetic dataset, and in order to avoid getting biased accuracy results, the program uses the holdout method on 20% of the dataset. In other words, it splits the dataset randomly into 80% training dedicated subset and a 20% test subset. The testing subset will be used ultimately for evaluation of the prediction capabilities of the machine learning built model [11].

2.4. Binary Logistic Regression Machine Learning Algorithm

This study leverages the use of the binary logistic regression machine learning algorithm. The latter analyzes the relationship between multiple input variables and a dependant categorical target variable. In our case, the target variable is either timestamp tampered files or not tampered. Let first define some concepts [12].

Odds of an event, such as the response variable y is a timestamp tampered, are the ratio of the probability p that the event will occur to the probability that it will not occur $1 - p$ (the event is the response y "not tampered").

$$odds(y = \text{"}timestamps\ tampered\text{"}) = \frac{p}{1-p} \tag{1}$$

Logistic regression models the natural log odds as a linear function of the input variables.

$$logit(y) = ln(odds) = ln(\frac{p}{1-p}) = \lambda + \sum_k \beta_k x_k \tag{2}$$

Where λ and β_k are the parameters of the logistic regression. Therefore, by simple algebraic manipulation:

$$p = Prob(y = \text{"}timestamps\ tampered\text{"}/x_k) = \frac{e^{\lambda+\sum_k \beta_k x_k}}{1 + e^{\lambda+\sum_k \beta_k x_k}} = \frac{1}{1 + e^{-(\lambda+\sum_k \beta_k x_k)}} \tag{3}$$

The logistic regression machine learning algorithm fits the regression coefficients λ and β_k in order to learn the relationship between the input variables and the target.

After tuning the hyper-parameters of the implementation of the machine learning algorithm for optimization of the results, the trained binary classifier can be dumped as a memory object for re-use in a production environment. The next subsection will concern the evaluation of the built model using multiple metrics.

3. Experimentation Results and Discussion

The strategy adopted to handle the imbalanced dataset is under-sampling. The figures 3 and 4 show the number of records of the dataset before and after the under-sampling.

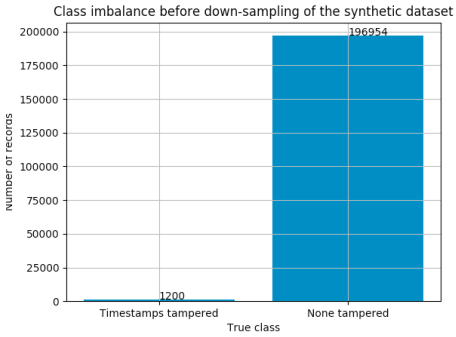


Fig. 3. Number of records for each class before down-sampling

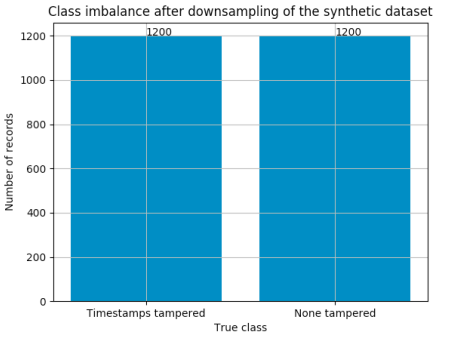


Fig. 4. Number of records for each class after down-sampling

We applied the binary logistic regression algorithm on the balanced dataset using the implementation of the algorithm available on the `scikit-learn` library [13]. We tune the parameter C (inverse of regularization strength) for values of [0.001, 0.01, 0.1, 1, 10, 10^2, 10^3, 10^4] and the parameter class_weights for 'balanced' or 'not balanced'.

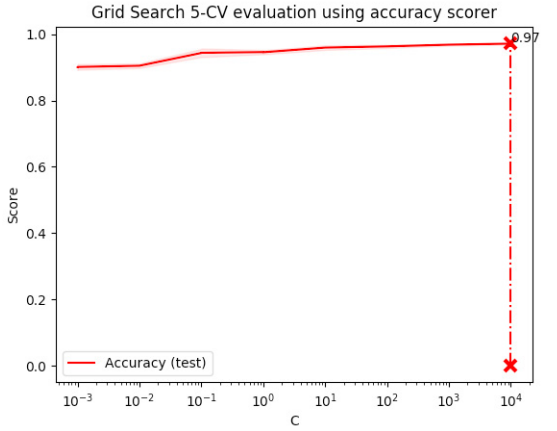


Fig. 5. A search for the best C parameter using accuracy scorer

The tuning of the hyper-parameters is done using an exhaustive grid search over all possible combination of C and *class_weights* hyper-parameters. To ensure the consistency of the results, we cross-validated the search 5 times. The used scorer is Accuracy. The results are displayed in figure 5. As can be seen, the best values obtained are $C = 10^4$ found with *class_weights* $='$ *balanced*$'$.

3.1. Model evaluation

Now, that we dispose of the best hyper-parameters, we can evaluate the performance of the built model using the confusion matrix. The computation of the confusion matrix reveals a class-wise accuracy of the predicted records with our algorithm set to its best parameters. The figures 6 and 7 show the normalized and the not normalized confusion matrix of the predicted results on the test subset of the dataset. From the figures, we can learn that the predicted timestamps tampered files that are truly timestamps tampered are 95% correctly predicted and 1% of the truly none tampered files have been classified as timestamps tampered files.

In order to visualize the performance of the built binary classifier, we plot the receiver operating characteristic curves (ROC curves figure 8) and precision-recall curves (PR curves figure 9) for different values of C.

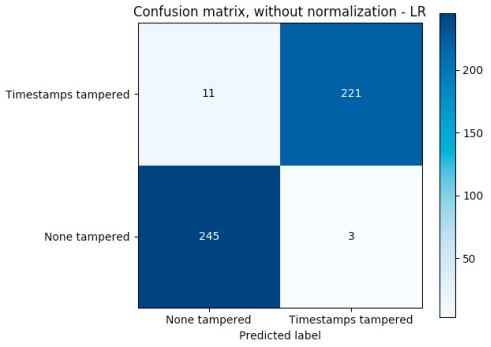


Fig. 6. Not normalized confusion matrix

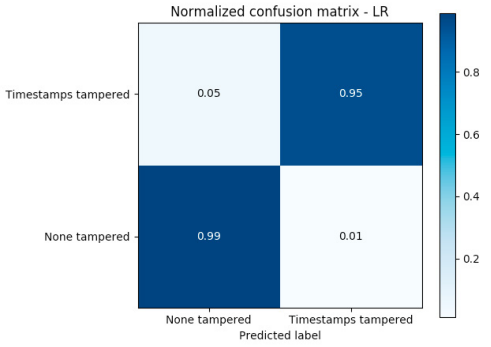


Fig. 7. Normalized confusion matrix

Figure 8 shows that all the areas under the ROC curves for different C values are near to 1, which reveal a good classifier for all the C values. However, the AUC of ROC curve with the $C = 10^4$ tend toward the ideal classifier far from the random hypothetical classifier designated by the middle black segment.

Figure 9 shows that all areas under the PR curves for different C values are near to 1, which means that the classifier is between ideal and good classifier.

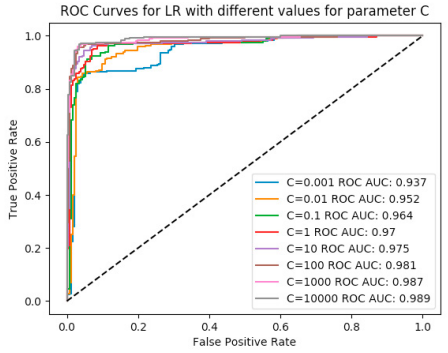


Fig. 8. ROC curves and AUC metric computed for C values.

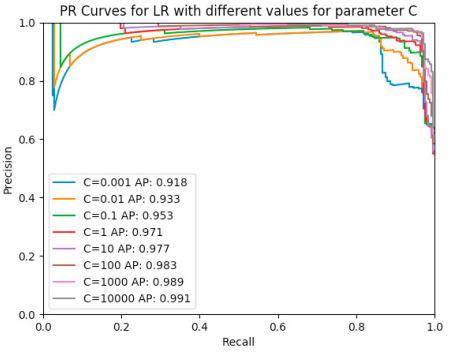


Fig. 9. Precision-Recall curves for different C values

Furthermore, in order to test the predictive ability of the built model on new data and if it generalizes well, we choose to 5 times cross-validate the results. The table 1 summarizes the mean value +/- standard deviation using multiple metrics: PR AUC, ROC AUC and log_loss. We observe a smaller value for log_loss which means better predictions results.

Table 1. Mean value and standard deviation of multiple evaluation metrics (PR AUC, ROC AUC and log_loss) cross-validated 5 times for the binary logistic regression with its best parameters.

Evaluation Metric	Mean value	Standard deviation	Observation
PR AUC	0.945	± 0.057	around 1
ROC AUC	0.965	± 0.0334	around 1
Log loss	1.22	± 1.15	small value

4. Conclusion

We presented an automatic approach for the detection of timestamps tampering using supervised machine learning. The built model was capable of predicting new tampered timestamps in the learned context. Our approach suggests that the binary logistic regression algorithm can learn efficiently from the engineered time features extracted from the NTFS filesystem. However, the proposed approach has some limitations motivating further research. The approach uses a single generated dataset and has been tested on a synthetic dataset. It could be possible to generate more data with different ratio of timestamps tampered files and none tampered. Another limitaton is that the approach was not yet tested on a real dataset, for instance: a dataset from a disk image compromised with timestomping capability malware.

References

[1] G.-S. Cho, A computer forensic method for detecting timestamp forgery in ntfs, Computers & Security 34 (2013) 36–46. doi:10.1016/j.cose.2012.11.003.

[2] J. Bang, B. Yoo, S. Lee, Analysis of changes in file time attributes with file manipulation, digital investigation 7 (3-4) (2011) 135–144. doi:10.1016/j.diin.2010.12.001.

[3] J. Bang, B. Yoo, J. Kim, S. Lee, Analysis of time information for digital investigation, in: 2009 Fifth International Joint Conference on INC, IMS and IDC, 2009, pp. 1858–1864. doi:10.1109/NCM.2009.258.

[4] F. Marturana, S. Tacconi, A machine learning-based triage methodology for automated categorization of digital media, Digital Investigation 10 (2) (2013) 193–204. doi:10.1016/j.diin.2013.01.001.

[5] D.-i. Jang, G.-J. A. H. Hwang, K. Kim, Understanding anti-forensic techniques with timestamp manipulation, in: 2016 IEEE 17th International Conference on Information Reuse and Integration (IRI), IEEE, 2016, pp. 609–614. doi:10.1109/IRI.2016.94.

[6] K.-P. Chow, F. Y. Law, M. Y. Kwan, P. K. Lai, The rules of time on ntfs file system, in: Second International Workshop on Systematic Approaches to Digital Forensic Engineering (SADFE'07), IEEE, 2007, pp. 71–85. doi:10.1109/SADFE.2007.22.

[7] S. Neuner, A. G. Voyiatzis, M. Schmiedecker, E. Weippl, Timestamp hiccups: Detecting manipulated filesystem timestamps on NTFS, in: Proceedings of the 12th International Conference on Availability, Reliability and Security, 2017, pp. 1–6. doi:10.1145/3098954.3098994.

[8] S. M. Ho, D. Kao, W.-Y. Wu, Following the breadcrumbs: Timestamp pattern identification for cloud forensics, Digital Investigation 24 (2018) 79–94. doi:10.1016/j.diin.2017.12.001.

[9] X. Du, M. Scanlon, Methodology for the automated metadata-based classification of incriminating digital forensic artefacts, arXiv preprint arXiv:1907.01421.

[10] B. Carrier, File system forensic analysis, Addison-Wesley Professional, 2005.

[11] H. Brink, J. W. Richards, M. Fetherolf, Real-world machine learning, Manning, 2017.

[12] H. Park, An introduction to logistic regression: from basic concepts to interpretation with particular attention to nursing domain, Journal of Korean Academy of Nursing 43 (2) (2013) 154–164.

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825–2830.